

Dropout: A Simple Way to Prevent Neural Networks from Overfitting

Abishai Srinivasan (471538)

Gurusaran Sivakumar (469304)

Keerthana Karunanithi (469312)

Varshini Gayathri Suresh (469038)

RWTH Business School, Aachen

Supervised By: Dr. Rossana Cavagnini

Agenda

- About the Paper
- Introduction to Neural Networks (NN)
- Underfitting, Overfitting, and the Notion of a Good Fit
- Classical Regularization Techniques
- Dropout: Concept, Intuition, and Mechanism
- Implementation of Dropout in NN
- Experimental Results Across Domains and Additional Insights
- Extensions: Dropout in Restricted Boltzmann Machines (RBM) and Gaussian Noise
- Conclusion and Takeaways

About the Paper

The Challenge:

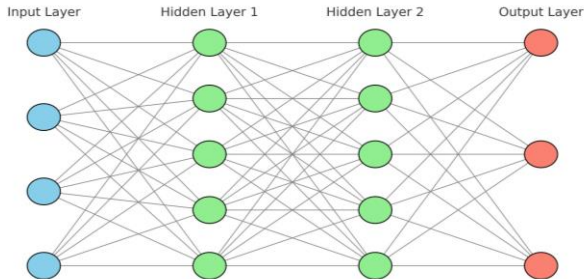
Overfitting in Neural Networks

Objective of the Paper:

Propose a simple and effective technique to prevent overfitting and improve generalization in deep neural networks without increasing computational cost.

What Is a Neural Network?

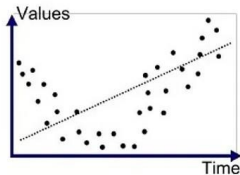
- A neural network is a layered computational model inspired by biological neurons.
- Each layer transforms the input using learned parameters (weights and biases).
- NN learn hierarchical representations via gradient-based optimization.
- Widely applied in image recognition, speech processing, and natural language tasks.



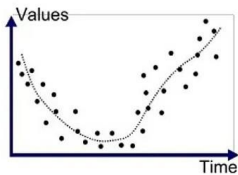
Source: OpenAI-generated image

Underfitting, Overfitting, and the Notion of a Good Fit

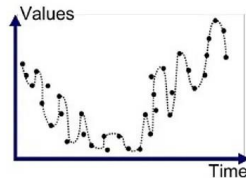
- Underfitting occurs when the model is too simplistic to capture data patterns.
- Overfitting arises when the model memorizes training data, including noise.
- A good fit balances data fidelity with generalization capability.
- Mitigating overfitting is critical in deep architectures with high model capacity.



Underfitted



Good Fit/Robust



Overfitted

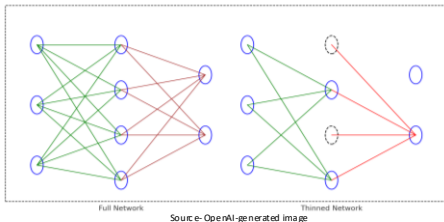
Source: [Medium](#)

Classical Regularization Techniques

Method	Mechanism	Strengths	Limitations
L1 Regularization	Penalize weight sum	Sparsity	May underfit in dense layers
L2 Regularization	Penalize weight squares	Stability and efficiency	Limited in deep models
Early Stopping	Halt on validation loss plateau	Simple to implement	Sensitive to noise, requires tuning
Data Augmentation	Input space expansion	Effective in vision tasks	Domain-specific applicability

Dropout as Efficient Model Averaging

- Model averaging is a proven technique for improving generalization by aggregating predictions from diverse models, but is computationally expensive, especially in deep architectures.
- Dropout offers an efficient alternative:
 - It is a stochastic regularization technique that randomly deactivates neurons during training.
 - Each forward pass effectively trains a thinned subnetwork drawn from the full architecture.
 - During inference, the full network is used with weights scaled by the dropout probability p to reflect the average contribution.
- Dropout helps prevent co-adaptation among neurons, leading to more robust, independently useful feature representations.



Dropout: Intuition and Mechanism

Intuition:

- Inspired by biological evolution and robust system design.
- Prevents neurons from relying on specific patterns or collaborations.
- Encourages independent feature learning and reduces co-adaptation.

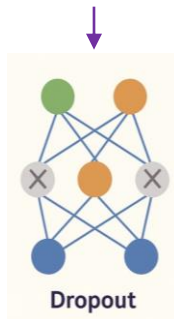
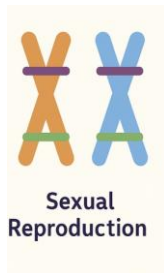
Training Mechanism:

During training:

- Each neuron is retained with probability p ; others are dropped temporarily.
- Each mini-batch sees a different random subnetwork.
- Gradients are computed only for active units.

At inference:

- Dropout is disabled.
- Weights are scaled by p to match expected activations.



Source: OpenAI-generated image

Related Work: Comparison Overview

Method	Mechanism	Limitations/Strengths
Denoising Autoencoders (DAE)	Adds noise to input data, reconstructs clean version	Unsupervised, focuses on input reconstruction, limited for classification
Dropout	Randomly drops hidden units during training, noise as regularizer	Scalable, efficient, approximates model averaging with shared weights

Model Description: Feedforward Network

Forward Pass without Dropout:

- Neuron activation is deterministic:

$$\begin{aligned}z_i^{(l+1)} &= \mathbf{w}_i^{(l+1)} \mathbf{y}^l + b_i^{(l+1)}, \\y_i^{(l+1)} &= f(z_i^{(l+1)}),\end{aligned}$$

$\mathbf{z}^{(l)}$ - Vector of inputs into layer l

$\mathbf{y}^{(l)}$ - Vector of outputs from layer l

$\mathbf{w}^{(l)}$ - Weights at layer l

$\mathbf{b}^{(l)}$ - Biases at layer l

- All units are active in every mini-batch.
- Network learns stable patterns without randomness in activation.

Forward Pass with Dropout:

- Apply dropout mask to each layer:

$$\begin{aligned}r_j^{(l)} &\sim \text{Bernoulli}(p), \\ \tilde{\mathbf{y}}^{(l)} &= \mathbf{r}^{(l)} * \mathbf{y}^{(l)}, \\ z_i^{(l+1)} &= \mathbf{w}_i^{(l+1)} \tilde{\mathbf{y}}^l + b_i^{(l+1)}, \\ y_i^{(l+1)} &= f(z_i^{(l+1)}).\end{aligned}$$

$\tilde{\mathbf{y}}^{(l)}$ - Thinned outputs

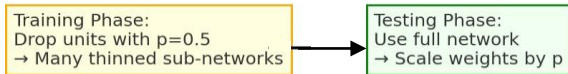
$\mathbf{r}^{(l)}$ - Vector of independent Bernoulli random variables

- Each mini-batch samples a random thinned network.
- Only active neurons contribute to output and weight updates.

What Happens at Test Time (With Dropout)?

- Use the full network, no neurons are dropped.
- Scale weights W by dropout probability p : $W_{test}^{(l)} = pW^{(l)}$
- This keeps the expected output the same as during training.

Dropout: Training vs. Testing Behavior



Backpropagation with Dropout

- For each mini-batch, a new dropout mask is sampled.
- **Inactive neurons:**
 - ✓ Output = 0
 - ✓ No gradient = no update
- **Active neurons:**
 - ✓ Fully participate in forward and backward pass

Important: All subnetworks share weights, so over time all neurons get trained fairly.

Techniques to Stabilize Dropout Training

Technique	Description	Benefit
Max-Norm Constraint	Restrict the norm of each incoming weight vector to be below a fixed constant c	Helps prevent large weights and improves generalization
High Momentum	Use momentum values in the range 0.95 to 0.99	Reduces the noise in the gradients caused by dropout
Learning Rate Annealing	Use a large initial learning rate and slowly decay it during training	Helps the network explore the parameter space less, and settle into a better minimum

Why these help: Dropout adds randomness; these help prevent instability and overfitting.

Using Dropout After Pretraining

- Pretrain layers using Restricted Boltzmann Machines (RBMs) or autoencoders.
- Before fine-tuning with dropout:
 - ✓ Scale pretrained weights by $1/p$
 - ✓ Use smaller learning rate for stability
- Prevents the network from “overwriting” learned features.

2-Phase Training Pipeline



Experimental Results : Overview

- To demonstrate the effectiveness of dropout across a wide range of tasks and data types.
- Evaluates dropout improves generalization across vision, speech, text, and biology tasks.
- Tested on diverse datasets: MNIST, SVHN, CIFAR-10/100, ImageNet, TIMIT, Reuters-RCV1, Alternative Splicing.
- Compares dropout-enhanced networks to traditional architectures and regularization techniques.

Experimental Results on TIMIT Speech Dataset

Task:

- Identify individual speech sounds from audio recordings using deep neural networks.

Dataset:

- 680 speakers across 8 American English dialects.
- 10 phonetically rich sentences per speaker.
- Clean, speaker-independent recordings.

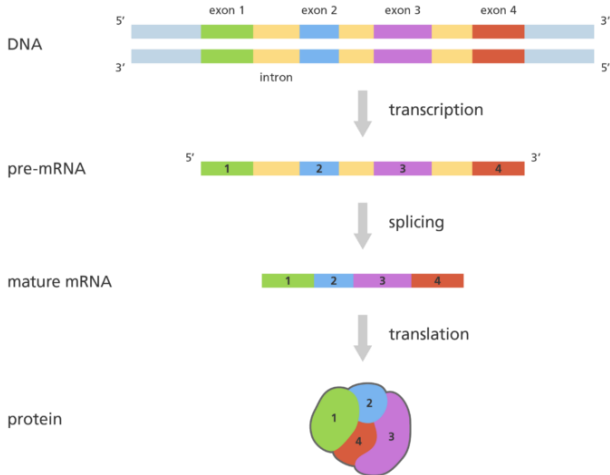
Model Setup:

- Input: 21-frame window of log-filter bank features.
- Deep Neural Networks (4, 6, 8 layers) with and without dropout.
- Dropout used both with and without DBN pretraining.

Takeaways:

- Dropout reduced phone error rate from 23.4% to 21.8% (6-layer NN).
- Best result (19.7%) achieved with dropout + DBN pretraining.
- Improved generalization without speaker-specific tuning.
- Effective on speech, not just images.

Biological Context Alternative Splicing



Source - [Your Genome](#)

Why Dropout Was Preferred over Bayesian Neural Networks

- Bayesian Neural Networks (BNN) model uncertainty by learning distributions over weights and averaging predictions across many models.
- While BNNs provide strong generalization in small-data settings (like RNA splicing), they are computationally expensive and hard to scale.
- Dropout, on the other hand, offers a simple and efficient alternative by randomly dropping neurons during training, approximating model averaging without heavy computation.
- Dropout was preferred in this work for its stability, scalability, and near-BNN-level performance making it practical for real-world tasks with limited resources.

Dropout for Biological Dataset

Task:

Predict alternative splicing outcomes from RNA sequence features.

Dataset:

- 3665 samples
- 1014 features
- 4 tissue types

Key Results:

- BNN: 623 bits
- Dropout NN: 567 bits
- SVM + PCA: 487 bits
- Regression + PCA: 463 bits
- Standard NN: 440 bits

Takeaways:

- Dropout performs close to Bayesian methods at a lower cost.
- Strong regularization in small-data settings.
- Scalable and practical for biological domains.

Experimental results on MNIST Experiment

Task:

Classify 28×28 grayscale handwritten digits into 10 classes.

Data Split:

60000 training, 10000 test samples.

Dropout Setup:

- Hidden layers: retain probability $p = 0.5$.
- Input layer: retain probability $p = 0.8$.

Key Techniques:

- Logistic & Rectified Linear Units (ReLU).
- Max-norm regularization.
- No early stopping required.

Key Results

- Standard NN (2×800, Logistic): 1.60%
- Dropout (3×1024, Logistic): 1.35%
- Dropout NN + ReLU: 1.25%
- Dropout NN + Max-norm: 1.06%
- Larger Net (2×8192, ReLU): 0.95%
- Dropout NN + Maxout (2×(5×240)): 0.94%
- Best (DBM + Dropout Finetuning): 0.79%

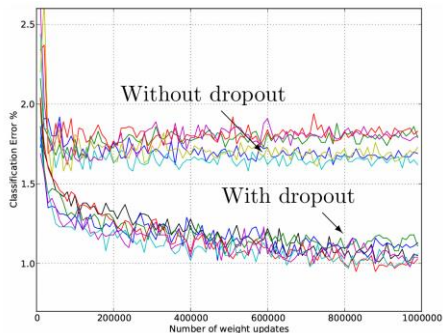
Method	Unit Type	Architecture	Error %
Standard Neural Net (Simard et al., 2003)	Logistic	2 layers, 800 units	1.60
SVM Gaussian kernel	NA	NA	1.40
Dropout NN	Logistic	3 layers, 1024 units	1.35
Dropout NN	ReLU	3 layers, 1024 units	1.25
Dropout NN + max-norm constraint	ReLU	3 layers, 1024 units	1.06
Dropout NN + max-norm constraint	ReLU	3 layers, 2048 units	1.04
Dropout NN + max-norm constraint	ReLU	2 layers, 4096 units	1.01
Dropout NN + max-norm constraint	ReLU	2 layers, 8192 units	0.95
Dropout NN + max-norm constraint (Goodfellow et al., 2013)	Maxout	2 layers, (5 × 240) units	0.94
DBN + finetuning (Hinton and Salakhutdinov, 2006)	Logistic	500-500-2000	1.18
DBM + finetuning (Salakhutdinov and Hinton, 2009)	Logistic	500-500-2000	0.96
DBN + dropout finetuning	Logistic	500-500-2000	0.92
DBM + dropout finetuning	Logistic	500-500-2000	0.79

Robustness of Dropout Across Architectures

- Tested on networks with 2 to 4 hidden layers, each with 1024 to 2048 units.
- Dropout consistently lowers test error, regardless of architecture.
- All hyperparameters kept fixed.
- Acts as a strong regularizer that is no tuning needed.

Training curves form two groups:

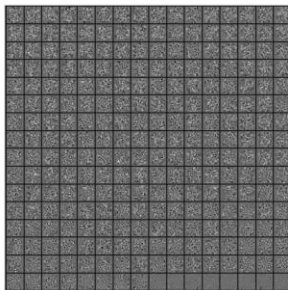
- With Dropout: Low error.
- Without Dropout: High error.



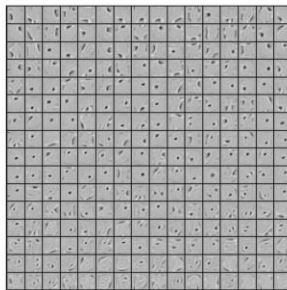
Source - [Dropout: A Simple Way to Prevent Neural Networks from Overfitting](#)

Effect on Features

- Dropout reduces co-adaptation between neurons.
- Forces each unit to be useful on its own.
- Learns robust, generalizable features.
- Visual outcome:
 - Without dropout: Fuzzy, overlapping filters.
 - With dropout: Clean edges, strokes, shapes.
- Improves representation quality in early layers.



(a) Without dropout

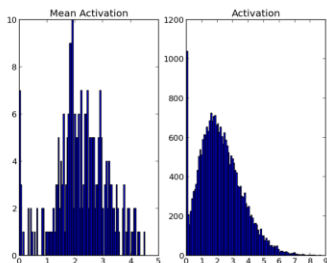


(b) Dropout with $p = 0.5$.

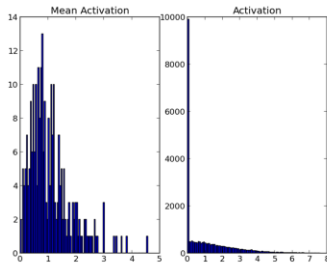
Source - [Dropout: A Simple Way to Prevent Neural Networks from Overfitting](#)

Effect on Sparsity

- Dropout induces sparse activations naturally.
- Results in:
 - Fewer neurons firing per input.
 - Lower mean activation per unit.
- Sparsity leads to:
 - Better generalization.
 - Reduced overfitting.
- Achieved without adding sparsity constraints.



(a) Without dropout

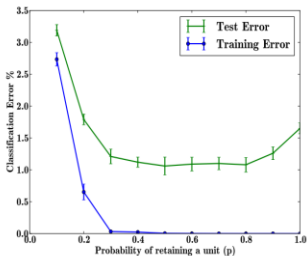


(b) Dropout with $p = 0.5$.

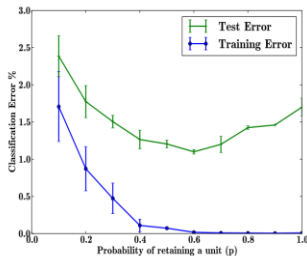
Source - [Dropout: A Simple Way to Prevent Neural Networks from Overfitting](#)

Effect of Dropout Rate

- Dropout rate is equal to probability p of retaining a unit.
- Two cases tested:
 - Fixed network size: lower p caused overfitting.
 - Fixed expected units (p times n): performance remained stable.
- Optimal range:
 - Hidden layers: p around 0.5 to 0.6.
 - Input layer: p around 0.8.
- Balances model capacity with noise-induced regularization.



(a) Keeping n fixed.

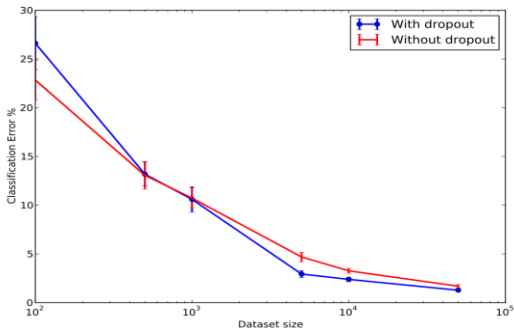


(b) Keeping pn fixed.

Source - [Dropout: A Simple Way to Prevent Neural Networks from Overfitting](#)

Effect of Dataset Size

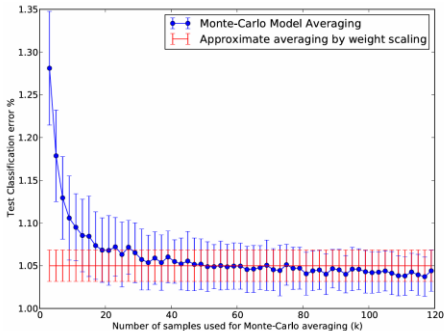
- Dropout performance varies with training data size.
- On very small datasets (<1000): minimal gains.
- On mid-sized datasets (5K-50K): significant improvements.
- On large datasets: still useful, but gains taper.
- Key Insight:
Dropout helps most when overfitting risk is high but data is big.



Source - [Dropout: A Simple Way to Prevent Neural Networks from Overfitting](#)

Monte Carlo Averaging vs Weight Scaling

- True model averaging involves sampling many dropout models.
- Too slow at test time, so use weight scaling instead.
- Approximate method:
 - Scale weights by p during inference.
 - Matches expected output from training.
- Results:
 - Scaling is nearly equal to Monte Carlo averaging with about 50 samples.
 - Much faster with similar accuracy.



Source - [Dropout: A Simple Way to Prevent Neural Networks from Overfitting](#)

Dropout for Text Dataset

Task:

- Classify news articles into 50 topics using a "bag of words" approach.

Dataset:

- Reuters-RCV1: Over 800,000 newswire articles.
- Subset used: 402,738 articles, 2000-word vocabulary, 50 classes.

Key Results:

- Standard neural net error rate: 31.05%
- Neural net with dropout: 29.62%
- Dropout reduced error, but the improvement was smaller than on image or speech data.

Takeaways:

- Dropout helps reduce overfitting even for large text datasets.
- The improvement is less dramatic when the dataset is very large and overfitting is less severe.

Dropout Restricted Boltzmann Machines

RBM:

- Two layers: visible (input) and hidden (features).
- Each visible unit connects to all hidden units; no connections exist within a layer.

Dropout in RBM:

- During training, some hidden units are randomly dropped (each kept with a set probability).
- For each example, only a random subset of hidden units is active; the rest are ignored.

Why Use Dropout?

- Forces hidden units to learn independently.
- Reduces overfitting and improves generalization.

How It Works

- Each training example uses a different random set of hidden units.
- This is like training many smaller RBMs with shared weights.

Model Description: Dropout RBM

Standard RBM Probability Distribution:

$$P(\mathbf{h}, \mathbf{v}; \theta) = \frac{1}{\mathcal{Z}(\theta)} \exp(\mathbf{v}^\top W \mathbf{h} + \mathbf{a}^\top \mathbf{h} + \mathbf{b}^\top \mathbf{v}).$$

- Defines the joint probability over visible ($\mathbf{v}$) and hidden ($\mathbf{h}$) units in a standard RBM.

Dropout RBM Joint Distribution:

$$P(\mathbf{r}, \mathbf{h}, \mathbf{v}; p, \theta) = P(\mathbf{r}; p) P(\mathbf{h}, \mathbf{v} | \mathbf{r}; \theta)$$

- Introduces the dropout mask ($\mathbf{r}$) and shows how the model combines the dropout pattern with the RBM distribution.

Dropout Pattern Probability:

$$P(\mathbf{r}; p) = \prod_{j=1}^F p^{r_j} (1-p)^{1-r_j}$$

- Specifies the probability of a particular dropout pattern, where each hidden unit is kept with probability p .

Learning Dropout RBMs: Feature and Sparsity Analysis

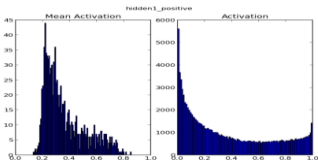
- Trained like standard RBMs, but randomly drop some hidden units for each training example.
- Only the remaining (active) hidden units are used in that training step.

Effect on Features:

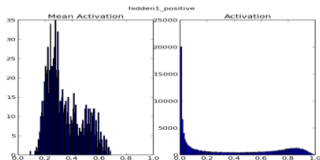
- Learns broader, more general features.
- Fewer unused (dead) hidden units compared to standard RBMs.

Effect on Sparsity:

- Produces sparser activations fewer hidden units are active for each input.
- Results in more efficient and distinct data representations.



(a) Without dropout



(b) Dropout with $p = 0.5$.

Source - [Dropout: A Simple Way to Prevent Neural Networks from Overfitting](#)

Marginalizing Dropout

What Is It?

- Mathematically averages over all possible dropout patterns, making the effect predictable and not random for each example.

Why Use It?

- Removes randomness from training, making it faster and more stable.
- Clarifies how dropout works as a regularizer to prevent overfitting.

Key Result:

- For simple models, acts like L2 regularization, with penalties scaled by feature variability.
- For complex models, can be approximated to explain or speed up dropout training.

Linear Regression with Dropout

- Dropout in linear regression acts like L2 regularization (ridge regression), helping prevent overfitting.
- The strength of the penalty for each weight automatically depends on how much its input feature varies in the data.

$$\underset{\mathbf{w}}{\text{minimize}} \quad ||\mathbf{y} - pX\mathbf{w}||^2 + p(1 - p)||\Gamma\mathbf{w}||^2$$

Where:

- $\mathbf{y}$ is the target vector.
- X is the input data matrix.
- $\mathbf{w}$ is the weight vector.
- p is the probability of retaining each input feature.
- $T = (\text{diag}(X^T X))^{1/2}$ is a diagonal matrix that scales the regularization for each weight according to the variability of each input feature.

Logistic Regression and Deep Networks

- For logistic regression and deep networks, finding a simple formula for marginalized dropout is difficult.
- Researchers use approximate methods to estimate dropout's effect.
- These approximations can speed up training and improve generalization.
- As networks get deeper, the accuracy of these approximations decreases.

Data Set	Architecture	Bernoulli dropout	Gaussian dropout
MNIST	2 layers, 1024 units each	1.08 ± 0.04	0.95 ± 0.04
CIFAR-10	3 conv + 2 fully connected layers	12.6 ± 0.1	12.5 ± 0.1

Source - [Dropout: A Simple Way to Prevent Neural Networks from Overfitting](#)

Gaussian Dropout

What is Gaussian Dropout?

- Applies multiplicative Gaussian noise to each neuron's output.
- Replaces 0/1 dropout mask with:

$$r \sim \mathcal{N}(1, \sigma^2) \Rightarrow h'_i = h_i \cdot r$$

r - Random multiplier from normal distribution

h_i – Original activation output

σ - Additional hyperparameter to tune

Why It Works:

- Keeps the mean activation unchanged (no scaling needed at test time).
- Introduces smooth, continuous randomness, unlike binary dropout.

Benefits:

- Simpler test-time behavior.
- Better suited for Bayesian models and variational inference.
- Sometimes outperforms standard dropout in practice.



Source - OpenAI-generated image

Conclusion

- Dropout is a powerful, domain-agnostic regularization technique. Prevents overfitting by training diverse sub-models.
- Performs well across:
 - ✓ Vision
 - ✓ Speech
 - ✓ Text
 - ✓ Biological data
- Easy to implement and scales to deep and wide networks.

References

Srivastava, N., Hinton, G., Krizhevsky, A., Sutskever, I., & Salakhutdinov, R. (2014). *Dropout: A Simple Way to Prevent Neural Networks from Overfitting*. Journal of Machine Learning Research, 15(1), 1929–1958.

Who Did What?

Name	Matriculation Number	Slides
Varshini Gayathri Suresh	469038	1-9 & 17-19
Gurusaran Sivakumar	469304	10-16
Keerthana Karunanithi	469312	20-27 & 35-38
Abishai Srinivasan	471538	28 – 34